(Dead-man-switch)

Permission-Based Wallet

A self-custodial Cardano wallet with shared permissions, spending limits, scheduled payments, and key-loss recovery

Shipped as **Epora**

Whitepaper

Project Catalyst Fund 11 — Cardano Use Cases: Concept

Author	Sandro Schaier
Platform	Cardano (Plutus v3 / Aiken)
Version	1.0 (June 2026)

Open-source. Built for the Cardano community.

Abstract

Wider adoption of Web3 and DeFi is held back by the lack of a reliable way to manage permissions. Traditional wallets can be backed up, seed phrases stored in vaults, extra hardware wallets provisioned, and access arranged for family members once they learn to use Web3; but each of these measures adds its own complexity and has its distinct problems.

Backups can be lost and require safe storage, hardware wallets risk breaking over time, and giving access to family members means trusting not only them but also their security measures.

Smart-contract and multi-signature wallets exist, for example for projects that manage large treasuries. While these tools spread risk and reduce the attack surface, they were built for large organizations and dedicated use-cases: the result is custom contracts, often not public, that favor function over usability. Ordinary users however have the same needs and few options that fit them.

This wallet, released as *Epora*, brings that treasury-management approach to individual users. Rather than ship a dedicated contract per feature and requirement, it threads the whole configuration through a single state-thread token (STT) datum and bounds every movement of funds with a two-validator handshake, so features compose as configuration on one reviewable contract that can be customized to the user's needs. Because funding works through ordinary transactions, it preserves interoperability with existing wallet infrastructure and a familiar dApp-like user experience.

The contracts and interface are open source; the implementation runs on the Cardano network.

Contents

Abbreviations 1

Glossary 1

1 Introduction 3

2 Background: The Extended UTXO Model 4

3 Design Goals and Threat Model 4
- 3.1 Design goals . . . 4
- 3.2 Threat model . . . 5

4 System Architecture 5
- 4.1 Features as configuration . . . 5
- 4.2 State-thread token and the two-validator handshake . . . 6
- 4.3 Why two contracts . . . 7
- 4.4 Receiving funds needs no datum . . . 8
- 4.5 Pinning the stake credential . . . 8

5 Permission and Recovery Model 9
- 5.1 Owners, allowances, and multi-signature . . . 9
- 5.2 The dead-man-switch . . . 9
- 5.3 Weighted-share beneficiary recovery . . . 10
- 5.4 Streaming payments and open settlement . . . 12

6 Formal Model 14
- 6.1 State and notation . . . 14
- 6.2 Streaming accrual and reserve . . . 15
- 6.3 Transitions and bounds . . . 16
- 6.4 The two validators . . . 16
- 6.5 Invariants . . . 17

7 Security Analysis 21

8 Limitations and Trust Assumptions 24

9 Illustrative Use Cases 29
- 9.1 Classical use cases . . . 29
- 9.2 Threats mitigated in practice . . . 29

10 Related Work 33

11 Implementation 33

12 Conclusion 35

Acknowledgments 36

References 36

Abbreviations

API Application Programming Interface

STT State-Thread Token

UTXO Unspent Transaction Output

Glossary

Wallet A traditional wallet is a program that stores a secret key and signs transactions to interact with a blockchain. Hardware wallets are a variation that keeps the secret on a dedicated device

Beneficiary In this context a beneficiary is any other person, represented by a traditional wallet, who may receive funds once the proof-of-life window (and any personal delay) has lapsed

Owner In this context an owner is a person with admin or sufficient agreeing multi-signature users with power over the wallet. By default the wallet has a single admin owner with broad access to the funds; this access can be restricted, for example by requiring a multi-signature quorum across several owners

Smart Contract Software whose execution is validated by a decentralized network, so its results can be trusted; deployed on a blockchain to offer on-chain functionality

Validator The EUTXO counterpart of a smart contract: a script attached to an output that runs when the output is spent and accepts or rejects the whole transaction

dApp A decentralized application: software, typically a web app, through which users interact with a blockchain and its smart contracts

Vesting Locking funds until a chosen date, before which the beneficiary (or owner) cannot access them

State Machine A computer-science model that describes a system as a set of states and the transitions between them; used here to model the wallet's reachable states

Proof of Life Proof of Life is the dead-man-switch heartbeat: a renewal of the wallet's unlock-time deadline. It is satisfied by any operator action that renews the window, or by a dedicated liveness-keeper renewal transaction through the dApp. If it is not provided before the deadline, beneficiaries may unlock

State-Thread Token (STT) A single native token, unique per wallet, whose continuing UTXO carries the wallet's authoritative configuration (the State) as its datum. Exactly one exists per wallet and it must be forwarded on every transition, so it is always the single source of truth for that wallet's rules

Streaming Payment A recurring payout that accrues linearly between a start and end date to a fixed payee. Before terminal recovery, a named stakeholder may settle it. After recovery opens, only an admin or the sole final beneficiary may settle it. The contract fixes the destination and caps the amount at the accrued value. The reserve prevents other spends from diverting accrued value

1 Introduction

Self-custody is the core promise of a public blockchain: the holder, and only the holder, controls the funds. That same property is its sharpest edge. A lost seed phrase, a hardware failure, a sudden death, or an incapacitating accident can put funds permanently out of reach, with no path to recovery. The very absence of an intermediary that makes self-custody trustless also removes the intermediary that, in traditional finance, handles inheritance, power of attorney, joint accounts, spending limits and other needs of regular users.

Existing answers are partial. A hardware wallet protects a key but does nothing if that key is lost or its holder is gone. Conventional multi-signature distributes trust but has no notion of time or inactivity, and remains operationally heavy. Custodial services restore familiar recovery only by taking custody away. What is missing is a wallet that keeps custody self-sovereign while encoding, on-chain, the everyday safety nets people expect: spending limits, joint control, automatic recurring payments, multi-wallet backup and a way for trusted parties to recover funds if the owner goes silent, with none of those parties able to seize funds while the owner is active.

This paper presents a permission-based (dead-man-switch) wallet for Cardano, funded under Project Catalyst [1, 2] and released to users as *Epora*. A single shared wallet is governed by an explicit, on-chain permission model rather than by one all-powerful key. An owner retains broad control while active. Beneficiary parties may recover funds only after the owner goes silent. Spenders may hold capped daily budgets, and recurring payouts settle to fixed recipients automatically. The whole configuration lives in one datum, carried by an STT, and every movement of funds is bounded by a two-validator handshake.

Contributions. This paper makes three contributions:

- A *features-as-configuration* architecture (Section 4) that replaces a combinatorial family of bespoke contracts with one reviewable contract whose behavior is selected by configuration. We treat that simplicity not only as a UX benefit but also as a security decision.
- A *permission and recovery model* (Section 5) with weighted-share, multi-beneficiary recovery and a proof-of-life dead-man-switch. It also includes per-day spending allowances and phase-aware streaming settlement. While a transaction's lower bound is before sole final-beneficiary recovery, listed users, stream payees, and unlocked beneficiaries may settle. Once that lower bound reaches the unlock boundary, only an admin or the final beneficiary may settle. The closest prior art (Section 10) has no such combination.
- An asset-based *security analysis* (Section 7) that states the threat model first, lists the protocol's assets, and gives the invariant defended for each; the design's limitations and trust assumptions are collected in Section 8. The model and these invariants are restated formally, with proof sketches, in Section 6.

2 Background: The Extended UTXO Model

Cardano uses the Extended UTXO (EUTXO) model [3], which underlies most design decisions in this work and is worth contrasting with the account model that many comparable wallets target.

In the account model popularized by Ethereum, global state is a set of accounts. Each is identified by a 20-byte address and holds a nonce, a balance, and, for contract accounts, code and persistent storage; a contract mutates its storage in place as transactions arrive [4]. In the UTXO model, there is no mutable per-contract storage: state is the set of unspent transaction outputs, and a transaction consumes some outputs and creates others. EUTXO extends each output with an arbitrary *datum* and guards spending with a *validator* script. That script sees the whole transaction (its inputs, outputs, value, signatures, and validity interval) and returns only accept or reject.

Two consequences matter here. First, EUTXO validation is deterministic and local: a script's verdict depends only on the transaction being validated, so the cost and outcome of a spend are known before submission and cannot be changed by unrelated on-chain activity. This suits a permission wallet, whose rules read directly off explicit inputs, outputs, signers, and time bounds. Second, with no in-place mutable storage, a wallet that needs persistent, evolving configuration must *thread* that state through a chain of outputs. The usual way to do this is a *state-thread token* (STT): a unique token whose continuing output always carries the current state as its datum, so the wallet's state is just the one output holding that token. However, this usually requires specialized transactions that encode a datum for every UTXO, which would break compatibility with existing UX flows. Therefore, while we adopt this pattern, described in Section 4, we also introduce a two-validator handshake to keep compatibility with existing wallet flows.

3 Design Goals and Threat Model

3.1 Design goals

G1 — Self-custody preserved.
No party may move funds outside the rules the owner configured; there is no privileged or centralized recovery, administrator override, or upgrade key over user funds.

G2 — Recovery is reachable.
A configuration that could permanently strand funds with no way back in must be rejected before it is ever created.

G3 — Authorization over restriction.
Safety derives from *who* may act and *when* (dead-man-switch timing), not from attempting to constrain where an authorized actor may send funds. This keeps the trusted path simple and avoids brittle policy-stacking.

G4 — Simplicity as a security property.
Unnecessary complexity is a leading root cause of vulnerabilities, and limiting complex logic is itself a decision that enables open external review [5]. The design therefore favors one small, auditable contract over a large or generated family of them.

G5 — Liveness without custody.
Recurring obligations must continue to settle even when the owner is offline or gone, without granting anyone custody. Streaming payments must stay funded, and funds accrued to third parties must remain withdrawable.

3.2 Threat model

A security claim is only meaningful against a stated adversary [5]. We assume an adversary who can submit arbitrary transactions, observe all on-chain data, choose transaction validity intervals freely within ledger rules, and supply their own inputs and outputs. The adversary may act as a permission-less third party, or control *some* keys (a single spender, a single beneficiary, a minority of multi-signature power), but not a quorum of owners. Cryptographic primitives and the Cardano ledger are assumed sound. Off-chain key custody at the edges (how a user stores their own key) is out of scope, as is transaction-level privacy. However, we provide ways to minimize these risks.

We do not apply a generic checklist such as STRIDE, which was built for software with few, mutually trusting parties and is weak at surfacing financial and collusion threats in cryptocurrency systems [6]. Instead we follow an asset-based method [6]: list the protocol's assets, state the secure behavior (the invariant) for each, and treat any violation as a threat. Section 7 carries out that analysis against the assets in Table 1.

Asset	Invariant to defend
Wallet funds (locked UTXOs)	Move only as a validated action permits
STT / State datum	Exactly one STT; state changes only as declared
Proof-of-life deadline	Each renewal advances it by at most one increment
Allowance counters	Spend bounded; daily reset cannot be forged
Streaming-payment reserve	Accrued-but-unpaid amounts cannot be diverted
Beneficiary share	At most a weighted share; nonfinal withdrawals are one-shot
Stake rights (rewards, delegation, vote)	Rest only at the intended stake credential
Wallet operability	Configuration caps and a persistent final recovery path

Table 1: Protocol assets and the invariant defended for each.

4 System Architecture

4.1 Features as configuration

A permission wallet must support many features (broad owner control, daily allowances, multi-signature, time locks, a dead-man-switch, streaming payments) and, above all, their *combinations*. Even with six features the number of distinct "and"/"or" combinations is

$$\sum_{k=1}^{6} \binom{6}{k} \times 2^{k-1} = 364,$$

and the growth is exponential: in general $\sum_{k=1}^{n} \binom{n}{k} 2^{k-1} = (3^n - 1)/2$, so a seventh feature already yields 1093. Building, testing, and auditing one customized contract per combination is therefore infeasible, and generating contracts on demand only moves that complexity, and its review burden, onto the user.

We take the opposite route. The wallet uses *one* contract whose features are *configuration*. The complete configuration (owners and their multi-signature threshold, beneficiaries, the proof-of-life timer, and the streaming payments) lives in a single *State* datum carried by the STT (Figure 1). Each feature is a field of, or an action on, that one object, so combinations are data a user selects rather than new code to audit. Under goal G4, collapsing the design to a small, auditable contract is a security and reviewability decision: it is what makes a full, open review tractable [5].

State	the STT datum: the entire wallet configuration
access.users[]	owners, spenders, liveness keepers: weight, per-day allowance ($\leq$ 10 users; $\leq$ 5 assets per allowance bundle)
access.multi_sig_threshold	θ, the weighted quorum for operator actions
access.beneficiaries[]	weight $\geq$ 1, optional unlock delay, full payout address ($\leq$ 15)
proof_of_life	unlock deadline d and renewal increment Δ
streaming_payments[]	payee, rate, start/end, paid-out ($\leq$ 15)
wallet_name	optional human label ($\leq$ 32 bytes; admin-only rename)
intended_stake_credential	the stake credential every wallet output must carry
last_non_admin_payout_at	shared stamp of the latest cadence-limited non-admin payout, payee cancel, beneficiary stream stop, final recovery, final exit, or final exact distribution. It anchors their cadence limit

Figure 1: The State datum carried by the STT. Every feature is a field, so feature combinations are data the owner selects, not new code to audit.

4.2 State-thread token and the two-validator handshake

The STT pattern, a multivalidator that both mints a unique state token (a Cardano native asset [7]) and governs every spend of the output carrying it, is established EUTXO practice [8, 9]; we do not claim it as novel. What this design adds is a deliberate *split* of enforcement across two validators that share one redeemer (the declared action) as their single source of truth (Figure 2):

- The **STT validator** authorizes every state transition and proves the declared action equals the true change to the State. For example, it checks that a declared allowance spend equals the actual decrease in the user's remaining allowance.

- The **wallet validator** is bound to the specific state-thread token and holds the funds. During spends it bounds how much value may leave, checking movement against the *same* declared action the STT validator just proved honest.

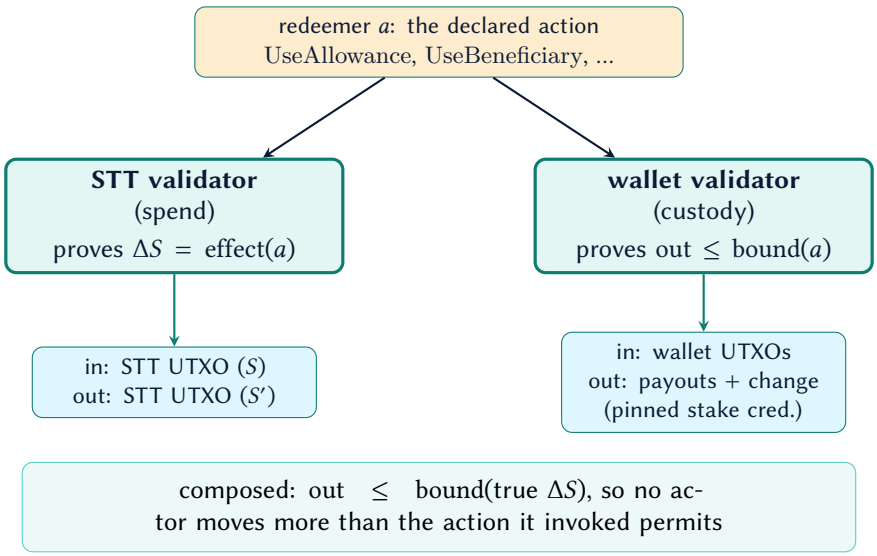

Figure 2: The two-validator handshake. Both validators read the *same* redeemer: the STT validator ties the declared action to the true state change, and the wallet validator bounds outgoing value by that action.

Guarantee

The **STT validator** proves *declared action = true state change*; the **wallet validator** proves *movement ≤ declared action.* Composed, **fund movement out of the wallet is bounded by the true change to the wallet's configuration**: no actor can move more than the action they invoked permits.

Because the two validators agree on one redeemer, there is no second, parallel encoding of intent to keep in sync, which is itself a simplicity gain (G4). For cost optimization the contract code stays in a permanent (spend always-fail) smart contract as a reference script, hosting the STT's script at a known location, so spends stay small without trusting a mutable pointer.

4.3 Why two contracts

Splitting the system into a *configuration* contract (the STT validator) and a *custody* contract (the wallet validator) separates two concerns that change at different rates and carry different risk. The wallet contract holds the funds but contains no permission logic of its own, while the STT contract reasons about permissions but never directly guards the bulk of the funds. Each can be read and audited on its own, and the funds live behind a small validator whose entire job is a value bound.

This separation also yields the property that makes funding the wallet straightforward. Because the custody contract places no conditions on what it receives, its address behaves, on the receiving

side, like any ordinary Cardano address. A deposit is therefore an ordinary payment, and any existing payor can fund the wallet without touching the STT UTXO. This also allows funds to be spread across parallel UTXOs, easing the size restrictions the ledger places on a single UTXO. The next subsection defines the mechanism precisely.

4.4 Receiving funds needs no datum

In the EUTXO model, a validator script runs only when an output it guards is *spent*, never when an output is *created* [3, 10]. There is no restriction on the outputs sent *to* a script address: anyone may pay a plain output, with no datum and no special construction, to the hash that identifies the wallet's custody contract, exactly as they would pay an ordinary key address [10]. The wallet's validator does not need to "accept" the deposit, because it is not consulted on the way in; it is consulted only later, when those funds are spent and the permission rules apply. The earlier claim that there was *no datum to get wrong* was incorrect. It confused the validator's unused datum argument with the ledger's witness requirements. A deposit needs no datum, but a sender can attach a datum hash. Spending that output still requires its preimage.[1]

The wallet now rejects hashed datums on continuing wallet outputs. It accepts outputs with no datum or an inline datum. Existing inputs remain eligible, but a hashed input still needs its preimage.

Guarantee

A permission wallet's address behaves, for the purpose of receiving, like any ordinary Cardano address. To fund it, a sender pays to the wallet's script address with no datum, no redeemer, and no awareness that a smart contract is involved at all. The permission rules engage only when funds leave.

For users, funding stays simple: a wallet can publish a single receive address and accept payments from any Cardano wallet, exchange, payroll system, or dApp, without those senders attaching a datum, integrating a library, or even knowing the destination is a smart contract. Shared treasuries, streaming-payment funding, and ordinary top-ups all work through this one address. The complexity of the permission model is paid for only by those who *spend*, never by those who *pay in*.

4.5 Pinning the stake credential

A Shelley address carries two independent parts: a *payment* credential that governs spending, and a *stake* credential that governs delegation and staking rewards [11]. The two are freely combinable: any payment credential may be paired with any stake credential. Combined with unrestricted receiving (Section 4.4), this opens a subtle gap. Anyone may pay to the wallet's payment credential under a stake credential *they* control, a so-called "Frankenstein" address. Spending such an output still requires the wallet validator, so the funds cannot be stolen. But the foreign stake credential

[1] The ledger adds the datum hash of each spent script output to its required witness set: getInputDataHashesTxBody.

would collect their staking rewards and direct their delegation and governance vote, and an ordinary address-based balance query would not even see them. The same drift can arise innocently, from a deposit made to an outdated address.

The State therefore records an *intended stake credential*, and the wallet validator requires every continuing wallet output to carry exactly it. Therefore no spend can re-home funds to a different stake credential. A value-preserving consolidation can return compatible stray inputs to the intended credential or repartition wallet value across continuing outputs. Consolidation has no fixed wallet-input or continuing-output count. The ledger byte-size and combined ExUnit limits decide which consolidation shapes fit. Consolidation also has no native-asset-row cap because it preserves aggregate wallet Value exactly. Operator *Use* has no five-asset cap. The intended credential changes only through a dedicated admin or quorum transition. Re-delegation is therefore deliberate and cannot occur as a side effect of another spend.

Guarantee

Every unit of wallet value that remains in the wallet after a spend comes to rest at the wallet's *intended* stake credential. Staking rewards, delegation, and governance weight on wallet funds thus stay under the wallet's control, changeable only by an authorized operator (admin or multi-signature quorum) through a dedicated transition.

5 Permission and Recovery Model

The wallet's complete behavior is a small, fixed vocabulary of transitions over the State. Table 2 summarizes, for each transition, who may invoke it and how much value may leave the wallet; the subsections that follow describe the security-relevant mechanics.

5.1 Owners, allowances, and multi-signature

An owner authorized through the admin or multi-signature path may move wallet value broadly (goal G3); the only floor is the streaming-payment reserve (Section 7). Multi-signature is *weighted*: each owner carries a signing power, and an action is authorized only when the summed power of the signing owners meets a configured threshold (for example, weight 3 of a total 5). A spender may draw only the available allowance: the remaining grant before reset, or the per-day grant after reset. The reset uses the transaction's earliest validity time, so a wider validity window cannot make it occur early. The rolling daily bound requires the configuration assumptions in Theorem 3.

5.2 The dead-man-switch

The proof-of-life timer drives the recovery model. While the owner provides proof of life before each deadline, whether by spending or by a dedicated liveness-keeper renewal, the wallet is "alive" and beneficiaries cannot act. A single renewal may push the deadline forward by at most one configured increment, so the window cannot be skipped arbitrarily far ahead in one transaction. If

Transition	Authority	Bound on fund movement
Operator use	admin or multi-signature	broad (authorization-gated; streaming-payment reserve still applies)
Update state	admin or multi-signature	none (no spend)
Remove access entry	admin or multi-signature	none (no spend; cheap, linear work)
Manage streaming payments	admin or multi-signature	none (no spend)
Set stake credential	admin or multi-signature	none (no spend; re-delegates wallet)
Renew proof of life	a liveness keeper	none (no spend)
Use allowance	the spending user	exactly the user's declared draw ($\leq$ remaining daily allowance; streaming-payment reserve applies)
Use beneficiary	one unlocked beneficiary	$\leq$ weighted share of (funds − reserve; streaming-payment reserve applies)
Exit beneficiary	one unlocked beneficiary	same weighted bound. Removes the actor permanently. Final exit requires no streams
Distribute beneficiaries	named beneficiary signs; all unlocked	one wallet input, exact shares, no streams, all rights retained
Pay streaming payment	phase-aware (see below)	accrued, unpaid value for the schedules included in a ledger-valid transaction. Each schedule names one asset and one fixed payee
Cancel streaming payment	payee; exact safe cutoff after final recovery opens	none (no spend, stops only the payee's own stream)
Stop beneficiary stream	one unlocked beneficiary	none (no spend, stops one stream at the exact safe cutoff)
Consolidate	operator or unlocked beneficiary	zero (aggregate wallet Value preserved exactly)

Table 2: The transition vocabulary: authority and the bound each places on outgoing value.

the owner goes silent and the deadline passes, recovery shifts from owners to beneficiaries. Figure 3 shows the reachable states; Figure 4 traces the same mechanics along a timeline.

5.3 Weighted-share beneficiary recovery

Once the timer (and a beneficiary's own optional delay) has lapsed, a beneficiary may withdraw at most its *weighted share* of the distributable pool: weight/(Σ weights) $\times$ (funds − reserve) per asset. Every beneficiary before the final one is removed from State in the same transition, so its withdrawal is one-shot. The final beneficiary owns all remaining beneficiary weight and stays in State. A recovery may select any wallet-input set that fits its action and ledger limits. It may leave continuing outputs for the streaming reserve or a smaller chosen withdrawal. The final beneficiary can retry with fewer inputs or a smaller draw after the shared thirty-minute cooldown. No transaction can prove that another UTxO does not exist or that no future deposit will arrive.

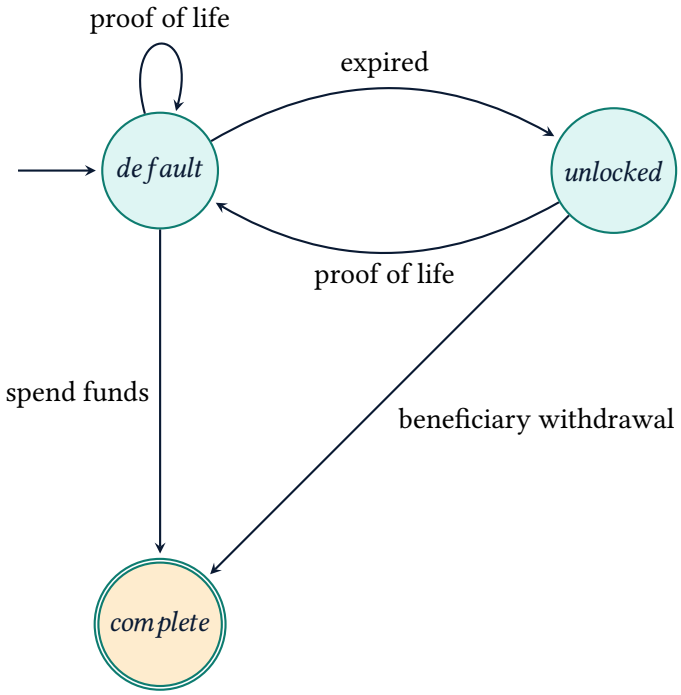

Figure 3: The dead-man-switch state machine: states reachable by the proof-of-life timer.

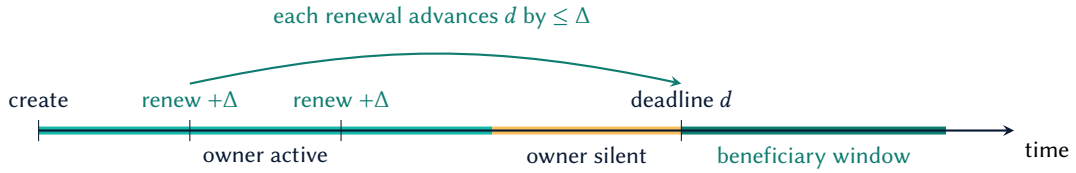

Figure 4: Proof-of-life timeline. While the owner renews, each renewal advances the deadline d by (usually) Δ and beneficiaries cannot act. Once the deadline passes with no renewal, each beneficiary may withdraw, after any personal delay of its own (e.g. a 21st birthday).

The State therefore has no final recovery marker. This recovery covers wallet UTxOs, not staking rewards.

The separate ExitBeneficiary action permanently removes the acting beneficiary, including the final one. It uses the same weighted withdrawal bound and streaming reserve. An earlier exit preserves the cadence stamp. A final exit requires an empty stream list and follows the existing thirty-minute cooldown. It may leave a terminal State with no remaining access path. Remaining funds and later deposits can then be inaccessible. Minting and configuration changes still require a reachable access path. Existing UseBeneficiary behavior remains available, including repeatable final recovery. Neither action freezes owner authority.

Each beneficiary now stores an explicit payout address for exact distribution. The address preserves both payment and staking credentials. It can name a key or a script. A receiving script must accept the wallet's existing tagged payout datum. Signing keys remain separate from this

destination. State ingress validates the address shape but does not reject payment credentials that match the wallet or state-token script. Exact distribution rejects both protocol payment credentials, including stake variants. A beneficiary intended for exact distribution must therefore use another payment credential. Existing weighted withdrawals and permanent exit do not use this destination. The added field changes the State schema and validator hashes; a four-field beneficiary record is not an in-place upgrade.

The separate DistributeBeneficiaries(i) action pays every remaining beneficiary from one wallet UTxO. All beneficiaries must be unlocked, and the named initiator must sign. The stream list must be empty. Stopping a stream does not erase its debt; settlement must remove it first. Existing weighted recovery remains available with its per-asset streaming reserve.

Let W be the sum of the remaining beneficiary weights. For every input asset x, including lovelace, the quantity q_x must satisfy $q_x w_b \bmod W = 0$ for every beneficiary b. Each recipient receives exactly $q_x w_b / W$ native units. The tagged output may contain more lovelace than its exact share to meet the ledger minimum. External inputs fund that extra lovelace and transaction fees. Each beneficiary has one distinct output at its complete configured address. The inline datum retains the existing OutputId(b, *transaction_id*, *output_index*) shape, bound to the consumed STT reference. A receiving script must accept this datum. There is no alternative datum envelope.

The action preserves every beneficiary and its weight. With multiple beneficiaries, it preserves the cadence stamp. With one beneficiary, it follows the existing thirty-minute cooldown and stamps the transaction's upper bound. It creates no continuing wallet output. An earlier value-preserving consolidation can separate indivisible remainders into another UTxO. Ledger limits still decide whether all recipient outputs fit. Asset assignments to named beneficiaries are not part of this action.

Preparation uses the existing Consolidate action. Let $g = \gcd(w_1, \ldots, w_n)$. A quantity divisible by W/g splits exactly across all current weights. Preparation can merge selected inputs or separate one divisible pool from a remainder. Every asset left out of the pool stays in the wallet. Both continuing outputs need their own minimum lovelace. Since consolidation preserves aggregate wallet Value exactly, those minimums must come from the selected wallet inputs. External inputs may pay the transaction fee. They cannot increase the continuing wallet Value in that consolidation. If the selected ADA cannot cover the output minimums, the beneficiary must select more wallet ADA or make a separate deposit first.

5.4 Streaming payments and open settlement

A streaming payment (surfaced in the product as a scheduled payment) accrues linearly between a start and end date to a fixed payee. While a transaction's lower bound is before sole final-beneficiary recovery, any listed user, stream payee, or unlocked beneficiary may settle accrued value. An admin may also settle. Once that lower bound reaches the unlock boundary, only an admin or the final beneficiary may settle. The contract fixes each destination and caps the value moved at the accrued, unpaid amount. Safety is structural, not identity-based (goal G5). This decouples a recipient's income from the owner's liveness. It also keeps accrued funds available to the recipient when the owner is offline or gone. The amount owed to active streaming payments forms a *reserve* that no other spend, not even an owner's, may draw below.

Each schedule names exactly one asset. The contract sets no fixed count of positive schedules per payout. One transaction may advance any batch that fits the ledger byte-size and combined ExUnit limits. A builder can retry a smaller batch. The schedules may name the same asset or different assets.

Settlements consume and re-create the single STT output, so a party able to crank at will could occupy the state thread and crowd out the owner's own transactions, including a proof-of-life renewal, at only fee cost. Two guards close this: *who* may settle, and *how often*.

Settlement is open but not anonymous. While a transaction's lower bound is before terminal recovery, it requires any listed user's, stream payee's, or unlocked beneficiary's signature. An admin is a listed user but uses a separate cadence-free branch. The payee arm keeps settlement reachable during this phase if every listed-user key is lost. Once that lower bound reaches the sole final beneficiary's unlock boundary, only an admin or that beneficiary may settle. This rule stops other stakeholders from advancing the cadence through payouts. Each payee still retains one exact terminal cancellation per payment.

Cadence is bounded for everyone but an admin payout. The State stamps each non-admin payout's upper validity bound. Payee cancellation, beneficiary stream stopping, final-beneficiary recovery, final exact distribution, and final permanent exit share this clock. The next cadence-limited action is accepted only when its earliest validity time lies at least thirty minutes past that stamp. Its validity window is capped at one hour. Each accepted action delays the other cadence-limited actions. Only an admin payout bypasses the limit, and it leaves the stamp untouched. Section 6 states the cadence this yields.

An earlier revision of this design made settlement fully permission-less and relied instead on a *progress* requirement — the streaming-payment set had to change — to stop no-op churn. That requirement was ineffective: one unit of progress satisfies it, so the churn it was written against remained available at fee cost. It has been removed, and the authority gate above replaces it.

Streaming payments are introduced at wallet creation or added afterwards by an operator (an admin or a multi-signature quorum) through the manage path; a newly added payment is always born unsettled, so it cannot fabricate already-paid progress. A fresh payment also cannot use the STT script payment credential, including through another stake-address variant. Mint obtains the credential from its runtime policy id. Management obtains it from the consumed STT input and checks only new payment ids. The contract therefore needs no self-hash parameter. The payout path settles accrued value and removes a payment once it has matured and been fully paid out. An operator can reschedule a payment's end date, moving it later to extend the payment or earlier to stop further accrual (never below the transaction's upper validity bound, so already-accrued value cannot be clawed back); an operator stops a payment by rescheduling its end date down, not by deleting the entry, so an existing payee can never be dropped. Existing payees are therefore untouchable except for this no-clawback reschedule, while the reserve accounting stays sound across additions because the per-asset reserve is recomputed from the live State's payee set on every spend rather than assumed fixed; the count cap (Section 6) bounds the set however it grew.

Each new payment also needs evidence that its asset exists. A consumed or reference input must contain a positive quantity of the exact policy ID and asset name. Every payment at mint is new. Management checks only output payment IDs absent from the input State. Existing forwarded or

rescheduled payments need no new evidence. A single base unit is enough, including when several new payments name the same asset.

A payment's own payee may also shorten it through a dedicated cancel path authorized by the payee's signature rather than operator authority. The new end must be strictly earlier than the old end. It cannot precede the payment's start or the transaction's upper validity bound, the ledger-safe proxy for "now". Before the payment starts, this rule permits the end to equal the start and creates a zero-lifetime obligation. Such an entry owes and reserves zero. The next payout must remove it, while fresh schedules remain strict positive-duration, zero-paid entries. The action cannot extend the payment, claw back accrued value, touch another payment, or move wallet value. It shares the global non-admin streaming cadence. The transaction uses a validity window of at most one hour and starts at least thirty minutes after the previous stamp. It writes its upper bound as the next stamp. There is no persistent cancellation flag. Before final recovery opens, the payee may shorten the same payment again after the cooldown. Once final recovery opens, the new end must equal the earliest safe cutoff. Without an intervening operator reschedule, a later cadence-valid interval starts after that end, so the same payment cannot cancel again. The 15-payment State cap gives 15 terminal cancellation slots. One preceding action may straddle the unlock boundary. The resulting payee-only delay from that boundary is at most 24 hours under the one-hour validity cap and thirty-minute cooldown. An admin or multi-signature quorum may later reschedule the payment. A script-credential payee cannot sign and has no self-cancel path. An operator manages that payment.

An unlocked beneficiary may stop one stream through StopBeneficiaryStream(i, j). The new end is exactly $\max(s_j, t_1)$ and must precede the old end. The action preserves the paid amount, rate, asset, payout address, start, and all other streams. Accrued debt remains payable. An already-ended stream goes directly to settlement. The stop also works for script payees because the beneficiary supplies the signature. It shares the thirty-minute cooldown and one-hour validity-window limit. Multiple stops therefore need separate cooldown slots. Existing settlement pays the debt and removes the stream. Stopping does not freeze owner authority: an operator may later reschedule a stream or renew proof of life.

6 Formal Model

This section makes the model of Section 5 precise: the state space, the transitions over it, the bound each transition places on outgoing value, and the invariants the two validators enforce, stated as theorems with proof sketches. The definitions mirror the Aiken types and on-chain checks, so each proof reduces to the predicate the contract evaluates. A transaction fixes a validity interval $[t_0, t_1]$ (its earliest and latest admissible times), and $D = 86\,400\,000$ is the number of milliseconds in a day.

6.1 State and notation

Definition 1 (Wallet state)**.** A wallet state is a tuple $S = (U, \theta, B, P, M, sc, \tau)$: a list U of user records, a weighted multi-signature threshold θ (possibly unset), a list B of beneficiaries, a proof-of-life setting $P = (d, \Delta)$ pairing the unlock deadline d with the renewal increment Δ (set together,

or absent when the dead-man-switch is unused), a list M of streaming payments, the intended stake credential sc, and a cadence stamp τ, the upper validity bound of the latest cadence-limited non-admin payout, payee cancellation, beneficiary stream stop, final-beneficiary recovery, final permanent exit, or final exact distribution (absent until the first). Each user u carries an admin flag, a renewal flag, an optional signing power, a per-day allowance perDay_u, a remaining allowance rem_u, and a reset deadline ρ_u; each beneficiary b carries a weight $w_b \geq 1$, an optional personal delay, and a full payout address. A streaming payment carries no cancellation marker: its end date alone determines when accrual stops, and equal start and end dates represent a zero-lifetime obligation. Every key or script credential hash stored in State is exactly 28 bytes, matching the ledger's Blake2b-224 credential width; this is checked at mint and every State ingress path rather than inferred from Aiken's byte-array aliases. Pointer stake addresses are rejected at the same boundary because new pointer addresses are unavailable in the deployed Conway-era protocol; payout addresses are therefore enterprise addresses or use an inline, width-checked stake credential. Asset identifiers are canonical too: ada is represented only by an empty policy and empty name, while a native asset has an exactly 28-byte policy id and an asset name of at most 32 bytes. The lists are bounded: $|U| \leq 10$, $|B| \leq 15$, $|M| \leq 15$.

The aggregate State caps also require $|U| + |B| \leq 15$. One user may list at most 10 wallet ids, and all users may list at most 15. One beneficiary may list at most 10 wallet ids, and all beneficiaries may list at most 15. Each per-day or remaining allowance bundle may contain at most 5 asset entries. The reserved allowance footprint satisfies $\sum_{u \in U} \left(|\text{perDay}_u| + \max(|\text{perDay}_u|, |\text{rem}_u|)\right) \leq 15$. This reserves room for a reset to replace a smaller remaining bundle. The wallet name contains at most 32 bytes. Each payment in M names exactly one asset.

The five-entry allowance limit bounds stored State data that later transactions must process. An ada entry counts toward this limit. The declared withdrawal also has a five-entry check and must equal the allowance decrease. Removing that check alone would not increase the available allowance. Wallet inputs and continuing outputs have no corresponding asset-count cap. For example, an input can contain ada and 20 native assets while a withdrawal takes only ada and token A. That draw uses two allowance entries. Every other asset and every unwithdrawn quantity must remain in the wallet. Authorization, streaming reserves, and ledger size and execution limits still apply. The current checks do not establish that five is the largest safe allowance size. A change to the stored allowance limit requires a separate budget review of later State transitions.

6.2 Streaming accrual and reserve

Definition 2 (Accrual)**.** For a streaming payment p with per-day rate r_p, start s_p, and end e_p, the amount accrued by time t is

$$\text{accr}_p(t) \;=\; \left\lfloor \frac{\left(\min(t,\, e_p) - s_p\right) r_p}{D} \right\rfloor .$$

Definition 3 (Per-asset reserve)**.** For asset x, the reserve sums accrued but unpaid amounts, with a one-unit buffer per unsettled payment. Here paid_p is the amount already paid to p's payee. A

payment's lifetime total is $\mathrm{accr}_p(e_p)$:

$$R_x(t) = \sum_{p \in M_x} \max\big(0,\ \mathrm{accr}_p(t) + 1 - \mathrm{paid}_p\big),$$
$$M_x = \{\, p \in M \ :\ \mathrm{asset}(p) = x,\ \mathrm{paid}_p < \mathrm{accr}_p(e_p) \,\}.$$

6.3 Transitions and bounds

Definition 4 (Transition). An action a is one of

RunOperator(π, κ), RenewProofOfLife, UseAllowance(δ),
UseBeneficiary(i), PayStreamingPayment(δ), Consolidate(π),
CancelStreamingPayment(j), ExitBeneficiary(i),
StopBeneficiaryStream(i, j), DistributeBeneficiaries(i),

with authority path π (admin or a multi-signature quorum, with an unlocked beneficiary also allowed for Consolidate), declared per-asset payout δ, beneficiary index i, streaming-payment id j, and operator kind κ, one of Use, UpdateState, ManageStreamingPayments, RemoveAccessIndex, or SetIntendedStakeCredential (the five operator rows of Table 2). A transaction tx drives a transition $S \xrightarrow{a} S'$, accepted exactly when (i) the authority required by a is met, (ii) $S' = \mathrm{Post}_a(S, tx)$, and (iii) the net value leaving the wallet is at most $\mathrm{bound}_a(S, tx)$. CancelStreamingPayment(j) requires payment j's payee signature. Its Post chooses an end e'_j satisfying $\max(s_j, t_1) \le e'_j < e_j$, leaves every other payment fixed, stamps $\tau' = t_1$, and moves no value. When sole final-beneficiary recovery is active, it also requires $e'_j = \max(s_j, t_1)$. Thus a pre-start cancel may have zero lifetime. Operator management may preserve or extend that existing zero-duration form. Its usual $s_j + 1$ stop floor applies to positive-duration inputs and fresh schedules. Cancellation is governed by the same finite-window and cooldown conditions as a non-admin payout. StopBeneficiaryStream(i, j) uses these same conditions, requires beneficiary i to sign after its effective unlock time, and always sets $e'_j = \max(s_j, t_1) < e_j$. It preserves all other stream fields and moves no wallet value. Here Post_a is the constrained post-state the action prescribes. We write ΔS for the actual change from S to S' and $\mathrm{effect}(a)$ for the permitted change Post_a describes. Thus condition (ii) is equivalently $\Delta S = \mathrm{effect}(a)$, the form the STT validator checks. Table 2 gives bound_a. Figure 5 shows one transition in full. While a transaction's lower bound is before sole final-beneficiary recovery, PayStreamingPayment requires a signature from any listed user, any payment payee, or any unlocked beneficiary. Once that lower bound reaches the unlock boundary, only an admin or the final beneficiary may authorize it. Theorem 9 gives its validity-interval and cadence conditions.

6.4 The two validators

Enforcement is split across two scripts that read the *same* redeemer a (Section 4.3, Figure 2).

Definition 5 (STT validator). V_S accepts $S \xrightarrow{a} S'$ only if the declared action equals the true state change, $\Delta S = \mathrm{effect}(a)$, exactly one state token is minted at creation, and exactly one continuing output carries it.

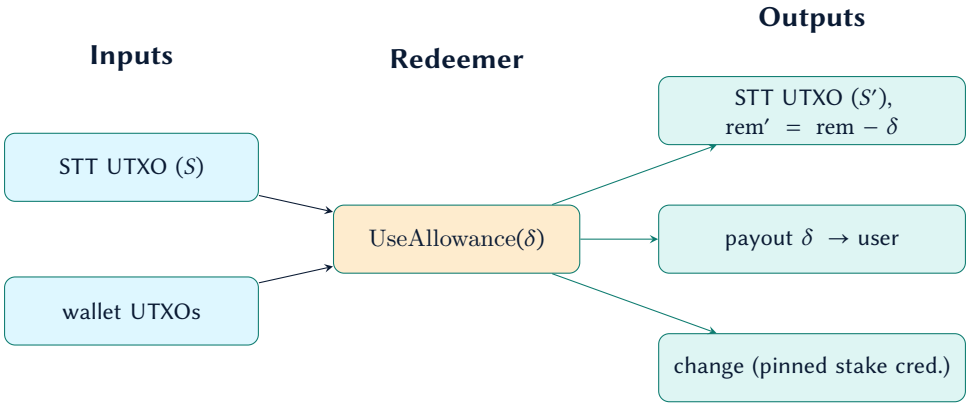

Figure 5: Anatomy of one transition, an allowance spend. The STT validator checks that the declared δ equals the drop in the user's remaining allowance; the wallet validator checks the payout is exactly δ and the change keeps the pinned stake credential.

Definition 6 (Wallet validator)**.** V_{W} accepts a spend only if, for every asset x, the continuing wallet value satisfies $\mathrm{out}_x \geq \mathrm{in}_x - \mathrm{bound}_a(x)$, where $\mathrm{bound}_a(x)$ is the asset-x component of $\mathrm{bound}_a(S, tx)$, and, for every action *except* PayStreamingPayment, the reserve floor $\mathrm{out}_x \geq \min(\mathrm{in}_x, R_x(t_1))$. The settlement is exempt because its outflow is already pinned per asset to the tagged payee outputs, so it cannot underfund a payee; applying the floor to it would instead deadlock an underfunded wallet, since the floor forbids every outflow once $\mathrm{in}_x \leq R_x(t_1)$ — including the only action that reduces R_x (Section 8, "Under-funded settlement"). Every continuing wallet output must additionally carry the credential *sc* and no reference script.

6.5 Invariants

Theorem 1 (Bounded movement)**.** *For every reachable S and accepting transaction with action a, the net value leaving the wallet is bounded, per asset, by* $\mathrm{bound}_a(S, tx)$*, taken against the true change to S.*

Proof sketch. V_{S} forces $\Delta S = \mathrm{effect}(a)$, so the declared action is the real one: for UseAllowance(δ) the declared δ equals the actual drop in rem_u, and so on. V_{W} independently forces $\mathrm{out} \geq \mathrm{in} - \mathrm{bound}_a$ for every asset. Both read the same a, so their composition bounds outgoing value by the action's true effect. □

Theorem 2 (Single state thread)**.** *From creation onward, exactly one state token exists, and the configuration is always the datum of the unique output carrying it.*

Proof sketch. Minting admits exactly one token; every spend forces exactly one continuing output bearing it and rejects a transaction presenting a second or none. The claim follows by induction on transitions. □

Theorem 3 (Allowance velocity). *Consider an interval with fixed user configuration and no operator allowance refill or reconfiguration. At its start, assume* $\mathrm{rem}_u(x) \le \mathrm{perDay}_u(x)$ *for every asset* x*. Within any half-open window of length* D *contained in that interval,* UseAllowance *draws at most* $\mathrm{perDay}_u(x)$ *of asset* x*. This bound holds for every accepted transaction validity interval.*

Proof sketch. The available allowance is perDay_u when $t_0 \ge \rho_u$, and rem_u otherwise. Each draw subtracts $\delta \ge 0$ and leaves $\mathrm{rem}'_u = \mathrm{available} - \delta \ge 0$. The initial assumption bounds the first grant, and every reset supplies at most perDay_u. Every draw sets $\rho'_u = \max(\rho_u,\, t_1 + D)$, at least D after its inclusion time. Thus a reset cannot supply another grant within D of any preceding draw. All draws in a half-open window of length D therefore share one grant and total at most perDay_u per asset.

A wider validity interval cannot cause an early reset because the lower bound t_0 must reach ρ_u. A later upper bound t_1 only postpones the next reset. The proof assumes that no operator changes the grants or reset deadline during the considered interval. □

Theorem 4 (Proof-of-life ceiling). *A* RenewProofOfLife *transition must strictly change the deadline, and advances it to at most one increment past the transaction's earliest time: the new deadline* d' *satisfies* $d' \ne d$, $t_1 \le d' \le t_0 + \Delta$ *and* $d' \ge d$ *— hence* $d' > d$*.*

Proof sketch. V_S accepts RenewProofOfLife only if the deadline changes and the three conjuncts hold. Combined with $d' \ge d$, the change can only be forward. Rejecting an unchanged deadline blocks a bit-identical no-op because the window conjuncts pass trivially when $d' = d$. *Correction:* this guard does not bound transaction frequency or set a minimum increase. A keeper can choose $d' = \max(d+1, t_1)$ whenever that value is at most $t_0 + \Delta$. The keeper can repeat this change against each accepted successor. This behavior can contest the single state thread at fee cost. It cannot move wallet value. The upper bound $d' \le t_0 + \Delta$ holds regardless of t_1. Thus, a wide window cannot skip the deadline arbitrarily far ahead. The same window check gates every transition that moves the deadline, except the operator's UpdateState. That action can reconfigure the timer from scratch (Section 8). □

Theorem 5 (Weighted-share recovery). *For* UseBeneficiary*, let* $W = \sum_{b \in B} w_b$ *over the beneficiaries present in S. An unlocked beneficiary* i *may withdraw, per asset* x*, at most* $\lfloor (w_i/W)\,\mathrm{pool}_x \rfloor$*, where* $\mathrm{pool}_x = \max(0,\, \mathrm{in}_x - R_x(t_1))$*. If* $|B| > 1$*, the transition removes* i *from S, so every nonfinal beneficiary acts once. If* $|B| = 1$*, the final beneficiary stays in S, satisfies the shared cadence, stamps* $\tau' = t_1$*, and may recover distributable value from separate wallet UTxOs through repeated transactions.*

Proof sketch. The bound is enforced division-free as $\mathrm{take}_x \cdot W \le w_i \cdot \mathrm{pool}_x$. Removing i when $|B| > 1$ gives the nonfinal one-shot property and re-normalizes the remaining weights. When $|B| = 1$, $W = w_i$, so the streaming-reserve floor alone gives the same bound. The final beneficiary remains in the output State. A final recovery may consume any wallet-input set that fits ledger limits. It may withdraw any amount that leaves the required reserve and may keep continuing wallet outputs. It can repeat with remaining inputs or a smaller draw after the shared cooldown. □

Theorem 6 (Exact distribution). *For* DistributeBeneficiaries(i), *a wallet spend consumes exactly one wallet input and leaves no continuing wallet output. Let* $W = \sum_{b \in B} w_b$. *For each input asset* x *and beneficiary* b, $q_x w_b$ *is divisible by* W. *The beneficiary's distinct tagged output contains exactly* $q_x w_b / W$ *native units and at least this share of lovelace. All beneficiaries are unlocked and remain in State. The stream list is empty.*

Proof sketch. The STT validator checks the initiator, unlock times, empty stream list, and State preservation. The wallet validator counts inputs and continuing outputs by payment credential. It checks exact divisibility and matches each payout to its full address and STT-bound beneficiary tag. Unique beneficiary ids prevent one output from satisfying two recipients. Summing the exact shares gives $\sum_b q_x w_b / W = q_x$, so splitting a pool across accepted transactions with unchanged weights introduces no rounding gain. Only external value can supply fees and extra payout lovelace. The STT leg alone cannot prove that a wallet input exists. The one-input rule applies when wallet funds are consumed. □

Theorem 7 (Streaming asset existence). *At mint, and for each new payment ID added through management, some consumed or reference input holds a positive quantity of that payment's exact asset.*

Proof sketch. The STT validator applies one shared predicate at both entry points. Mint passes an empty previous payment list. Management passes the consumed State's payment list. For each output ID absent from that list, the predicate checks the input Values by policy ID and asset name. It accepts only a positive quantity. It does not inspect transaction mint or output Values. □

Theorem 8 (Streaming reserve). *Every value-moving spend except* PayStreamingPayment *retains at least the lesser of each asset holding and its reserve:* $\text{out}_x \geq \min(\text{in}_x, R_x(t_1))$. *Such a spend cannot move accrued, unpaid value out. It also cannot reduce an asset that is already below its reserve.*

Proof sketch. V_W requires $\text{out}_x \geq \min(\text{in}_x, R_x(t_1))$ per asset on every non-settlement spend. Settlement is exempt because its exact outflow is pinned to tagged payees. The guard is point-in-time and covers accrual only to t_1. Section 8 records the consequence. □

Theorem 9 (Settlement cadence). *Let* $C = 1\,800\,000$ *(thirty minutes) and* $L = 3\,600\,000$ *(one hour). Terminal recovery is active when one beneficiary remains and* t_0 *reaches its effective unlock time. Every* PayStreamingPayment *must declare a finite validity interval. Before terminal recovery, it requires a signature from any listed user, stream payee, or unlocked beneficiary. When terminal recovery is active, it requires an admin's or the final beneficiary's signature. Every* CancelStreamingPayment *must declare the same bounded interval and carry the target payee's signature. During terminal recovery, it must also set* $e'_j = \max(s_j, t_1)$. *Every final-beneficiary recovery also declares this interval. Every non-admin payout must satisfy* $t_1 - t_0 \leq L$ *and* $t_0 \geq \tau + C$. *Every payee cancellation, beneficiary stream stop, final-beneficiary recovery, final permanent exit, and final exact distribution satisfies the same conditions. Each cadence-limited action stamps* $\tau' = t_1$. *An admin payout leaves* τ *unchanged. The contract sets no fixed positive-schedule count per payout. The ledger byte-size and ExUnit limits decide each batch. The true on-chain times of consecutive cadence-limited actions then lie at least* C *apart. No such action defers the next admissible one by more than* $L + C$ *past its own earliest bound.*

Without an intervening operator reschedule, a terminal cancel cannot repeat for the same payment. The 15-payment cap gives 15 terminal actions. One pre-terminal action may start before the unlock boundary and stamp after it. The conservative payee-only delay from that boundary is therefore $(15 + 1)(L + C) = 24$ *hours.*

Proof sketch. A transaction validates only inside its declared interval, so its true time lies in $[t_0, t_1]$. The previous cadence-limited action stamped τ with its t_1, at or after its true time. The next such action needs $t_0 \geq \tau + C$, so its true time follows the previous one's by at least C. The bound choice makes the gate non-gameable. Gating on the *earliest* bound keeps a wide window from faking an elapsed cooldown. Stamping the *latest* bound keeps a past-dated stamp from shortening the next one. The cap $t_1 \leq t_0 + L$ bounds the stamp's overshoot. The next action is therefore admissible by $t_0 + L + C$. An admin payout leaves τ unchanged, so it defers nothing. The phase-aware authority check prevents another stakeholder from advancing the clock after terminal recovery opens. □

Theorem 10 (Recovery reachability)**.** *Every state reachable through a configuration transition retains an authority path: a signable admin record (one carrying at least one wallet key), a satisfiable multi-signature quorum, or a signable beneficiary. Every* UseBeneficiary *transition also retains a beneficiary path because it never removes the final beneficiary. Exact distribution preserves the entire beneficiary list. The explicit* ExitBeneficiary *action is an exception: a final exit may deliberately end all recovery access.*

Proof sketch. Configuration validation enforces this at creation and on every UpdateState, and the linear-cost RemoveAccessIndex re-checks it. Hence an admin-less, recovery-less state is unreachable by configuration. UseBeneficiary removes an acting beneficiary only while another beneficiary remains. It preserves the final beneficiary in State. The final beneficiary can therefore recover separate current or future wallet UTxOs through repeated transactions. Each recovery may select any wallet-input set that fits ledger limits and may leave reserve-aware change. No transaction can prove that another UTxO does not exist or that no future deposit will arrive, so the State has no final recovery marker. Signability is enforced for all three configuration arms uniformly: the admin arm counts an is_admin record only when it carries a wallet key, matching the multi-signature arm's positive-power-with-wallet rule and the beneficiary arm's wallet requirement. A wallet-less admin record can never satisfy the operator gate, so it does not count as a path. Which *key* an owner uses for a signable admin, and its custody, remain the user's risk (Section 8). Recovery through wallet UTxOs does not include staking rewards, which remain operator-only. □

Theorem 11 (Stake-credential pinning)**.** *Every continuing wallet output rests at the intended stake credential sc, which changes only through the dedicated operator transition.*

Proof sketch. V_{W} rejects any continuing wallet output whose stake credential differs from *sc*, the settlement crank included; *sc* is writable only by SetIntendedStakeCredential under admin or quorum authority. □

7 Security Analysis

Following the asset-based method of Section 3, we state the invariant defended for each asset and how the validators enforce it. Each is backed by a regression test that reproduces the corresponding attack and asserts the attack fails. Where this section argues from the protocol's assets and adversary, Section 9.2 maps the same mechanisms onto the practical loss scenarios they contain.

Single state thread.
Minting forces exactly one state token, and every spend forces exactly one continuing output carrying it; a transaction presenting a second token at the script, or none, is rejected. The State is therefore always unambiguous.

STT-output value permissions.
Every accepted STT spend preserves the current wallet STT policy, asset name, and quantity one. Every otherwise-valid action may increase ada at the continuing STT output through external funding. Only an admin or a multi-signature quorum using $\mathrm{RunOperator}(\mathrm{Use})$ may reduce ada or add or remove assets under other policies. All other actions preserve every native-asset quantity.

Bounded movement.
By the two-validator guarantee, outgoing value is bounded by the declared action, and the declared action is proven equal to the true state change. An owner spend is broad but still floored by the reserve; allowance, beneficiary, streaming, and consolidation spends are each bounded exactly as in Table 2.

Allowance velocity.
The reset uses the transaction's earliest validity bound, and the next deadline uses its latest bound. These checks prevent an early reset through a chosen validity interval. Theorem 3 states the daily bound under its initial-grant and fixed-configuration assumptions. Configuration and allowance use both enforce the reserved reset footprint.

Proof-of-life ceiling.
A renewal's new deadline must lie at or beyond the transaction's latest bound and at most one increment past its earliest bound, so a wide validity window cannot push the deadline more than one increment ahead.

Streaming reserve, per asset.
A discretionary value-moving spend with active obligations must retain, for each asset it spends, at least the lesser of what the wallet holds and what it owes for accrual up to the transaction's upper validity bound. This is a point-in-time reserve, not the full forward schedule. Settlement is exempt because its exact outflow is pinned to tagged payees. Per-asset accounting means a spend in one asset is never blocked by a payment in another.

Payout integrity.
A streaming settlement may only increase paid-out progress, must route each asset to the configured payee address carrying the matching output tag, and, for a non-ada asset, the tagged outputs must sum to *exactly* the computed accrued delta. The continuing STT output preserves every native-asset quantity and may increase ada. This permits a larger State datum to meet its minimum-ada requirement. For an ada-denominated stream the tagged outputs must sum to *at*

least the delta: because the payout asset is ada and every output must itself carry a minimum ada amount, an exact-delta payee output below that minimum is unconstructible, so the crank tops the payee output up to a valid amount with its own ada (see Section 8, "ADA settlement granularity"). This relaxation does not widen what the wallet can lose: the wallet's *net* outflow is independently pinned to exactly the delta, and no output other than a wallet self-return, the *full-address* state-token continuation, or a correctly-tagged payee output may carry the payout asset, including lovelace. Full-address matching matters because another output may share the state validator's payment credential under a different stake credential without being the token-bearing continuation; such an output is not exempt. This closes the double-satisfaction pattern where an external input funds the payee while wallet value leaks elsewhere. With several ada streams settled in one transaction, wallet-sourced ada may be split across their configured payees. For a payee that does not share the wallet payment credential, its tagged output still contains at least its settled amount. A self-addressed stream is the source-attribution exception: its gross tagged wallet output can satisfy the payout floor while the wallet-funded ada delta reaches another configured payee or transaction fees. The total wallet loss remains pinned to the validated delta. Section 8 documents this accepted configuration risk. As the settlement crank is the wallet spend whose submitter need not be the owner, it is also bounded against UTxO-layout griefing: the number of continuing wallet outputs may not exceed the number of wallet inputs it consumes, so a cranker can consolidate or preserve the wallet's UTxO count but never fragment it into ever more dust; the owner can re-split through any authorized spend. For the same reason no continuing wallet output may carry a reference script: an attached script is charged per byte on every later transaction that consumes the UTxO and counts against the per-transaction reference-script limit, so without the ban a cranker could bloat the wallet's UTxO set and raise the cost of every later recovery spend. This mirrors the identical ban on the state-token output.

Settlement cadence.

While a transaction's lower bound is before sole final-beneficiary recovery, a payout needs any listed user's, stream payee's, or unlocked beneficiary's signature. Once that lower bound reaches the unlock boundary, only an admin or the final beneficiary may sign a payout. A cancellation always needs its target payee. Every non-admin payout, payee cancellation, beneficiary stream stop, final-beneficiary recovery, final permanent exit, and final exact distribution must start at least thirty minutes past the previous shared stamp. Each uses a validity window of at most one hour. The transaction's *earliest* bound gates the limit, and its *latest* bound becomes the next stamp. Before terminal recovery, a payee may shorten the same end again after the cooldown. During terminal recovery, it must use the exact safe cutoff. Without an intervening operator reschedule, each payment can cancel once. One pre-terminal action may straddle the unlock boundary, so the conservative payee-only delay from unlock is 24 hours.

Recovery reachability.

At creation and on every state update, the configuration must retain a reachable admin, multisignature, or beneficiary path. A wallet-less admin cannot satisfy the operator gate, so it does not count. Recovery through UseBeneficiary removes only nonfinal beneficiaries. It keeps the final beneficiary in State so that it can recover separate wallet UTxOs through repeated transactions. ExitBeneficiary is the intentional permanent-exit exception and may remove the final beneficiary while wallet funds remain.

Credential aggregation.
Wallet value is aggregated by payment credential, not full address, so a beneficiary cannot multiply its share by spreading wallet outputs across stake-credential variants of the same script in one transaction.

Continuing wallet output shape.
The shared output guard rejects DatumHash on every output that returns to the wallet payment credential. It accepts NoDatum and InlineDatum. A stakeholder who settles a streaming payment therefore cannot make wallet change depend on a private datum preimage. The guard does not restrict input datums or outputs to other payment credentials. Legacy hashed inputs still require their preimages. The evidence map links this rule to rejection and compatibility tests in validators/wallet_datum_tests.ak.

Stake-credential pinning.
Every continuing wallet output must carry the State's *intended stake credential* (Section 4.5). No spend, the settlement crank included, can re-home wallet funds to a foreign ("Frankenstein") stake credential to capture their staking rewards, delegation, or governance vote, or to hide them from address-based discovery; only an admin or quorum may change the intended credential, through a dedicated transition.

Bounded execution cost.
The State permits at most 10 user records, 15 beneficiary records, 15 records across both lists, and 15 streaming schedules. Each user may list at most 10 wallet ids, with 15 across all users. Each beneficiary may list at most 10 wallet ids, with 15 across all beneficiaries. Each allowance bundle may contain at most 5 asset entries. The reserved allowance footprint across all users is at most 15. The wallet name contains at most 32 bytes. Wallet spends have no fixed wallet-input count. The ledger byte-size, combined ExUnit, and action-specific Value limits decide how many inputs fit. Wallet inputs, outputs, and aggregate Values have no native-asset count cap. UseBeneficiary still bounds each withdrawn asset by the actor's weighted share. UseAllowance bounds its declared draw through the five-entry allowance bundle, which includes any ada entry. Exact subtraction preserves every asset outside that draw, and the streaming reserve check still applies to withdrawn assets. Ledger byte size and combined ExUnits still limit their input and output shapes. UseAllowance, UseBeneficiary, ExitBeneficiary, and PayStreamingPayment also require continuing wallet outputs not to exceed consumed wallet inputs. The contract sets no fixed general transaction input, general output, ordinary redeemer, or positive-schedule payout count. Serialized byte size and combined ExUnits decide whether each remaining transaction shape fits. Signer and governance-purpose bounds remain authority rules. Access growth uses UpdateState, while schedule growth uses ManageStreamingPayments. An addition above a State cap fails before the new State becomes reachable. A linear-cost removal can shrink a decodable legacy access list while each removal fits the current execution limits. The repository's capped-list fixtures pair the user, combined-access, wallet, allowance, and stream caps with high-width scalar fields and a minimum useful action. They use five beneficiaries. They measure named STT and wallet legs. They do not prove every optional field encoding, transaction topology, or co-spent external validator combination.

The authorization-gate stance (G3) is a deliberate trade-off, taken up in Section 8: a quorum of owner keys can move funds freely down to the reserve, because the design's safety rests on the

authorization gate and dead-man-switch timing rather than on on-chain spend restrictions.

8 Limitations and Trust Assumptions

This section lists what the design does not protect against and the trust assumptions it makes.

Trust assumption

The dead-man-switch protects against the owner going *silent*, not against a compromised key while the owner is active: anyone holding sufficient owner authority can spend and reset the timer. Liveness also depends on the off-chain wallet renewing proof of life on the owner's behalf. The on-chain timer enforces the deadline, but choosing a sensible timeframe and trustworthy beneficiaries remains the owner's responsibility.

Broad operator spend (by design, G3).
A quorum of owner keys can drain the wallet down to the streaming reserve. The mitigations are the dead-man-switch, the recovery-reachability invariant, multi-signature thresholds, and trusted transaction construction, not on-chain destination or amount limits. Narrowing the operator envelope would be a product change, not a security fix.

Allowance grants and operator changes.
Configuration validates the remaining and per-day allowances independently. The initial remaining grant may exceed the per-day grant for an asset. An authorized operator may also refill or reconfigure a user's allowance through UpdateState. These are accepted grants, not unauthorized withdrawals. The earlier unconditional daily-bound claim omitted these cases. Theorem 3 now states the required starting bound and excludes operator changes during the considered interval.

Wallet-less allowance succession.
A signed allowance user may submit a non-empty UseAllowance transition without consuming a wallet input. The STT-only form reduces the user's remaining allowance but moves no wallet value. It may repeat against each recreated STT until no positive effective allowance remains. A later reset supplies the next configured grant. Each call needs external fee funding and has no cadence rule.

Positive-power records require distinct credentials.
At mint and full state update, each payment key hash may appear in at most one user record with positive multi-signature power. Each signed record contributes its configured weight once. Records with no power or zero power may share credentials, and a user credential may also identify a beneficiary. Distinct credentials do not prove distinct people. One person who controls several keys can still activate several weighted records. The frontend applies the same rule before transaction construction.

Admin-only wallets are permitted.
A wallet may be configured with a sole admin and no other recovery path; losing that key is then an accepted user risk, exactly as for any single-signer wallet. The reachability invariant only guarantees that an *admin-less* wallet always retains a non-admin way back in.

Shape, not timing, of recovery.
The contract rejects only genuinely bricked configurations; a beneficiary whose unlock lies far in the future is accepted on-chain and flagged off-chain. A trusted liveness keeper can defer beneficiary unlock indefinitely while it keeps renewing, and an operator (admin or quorum) can push the deadline far forward in a single reconfiguration: the one-increment cap binds only the heartbeat renewal paths, not the full state-update path, which must be able to (re)configure the timer from scratch.

Authorized proof-of-life renewal contention.
The exact no-op guard does not set a minimum deadline increase or renewal cadence. A keeper can advance the deadline by one unit per accepted successor while each value remains inside the renewal window. Each successor can invalidate pending operator or beneficiary transactions that use the prior state token. The keeper pays every fee and cannot move wallet value through this action. The 2026-09 security review accepts this availability risk because the keeper is a trusted liveness role.

Streaming payments are additive and end-date governed.
An operator (admin or quorum) may add streaming payments to a live wallet or configure them at mint. Each addition is unsettled, and the count cap bounds the set. Management cannot delete an existing payment or change its immutable fields. It can move the end later, or earlier down to the transaction's upper validity bound. This rule protects already-accrued value. A payment's own payee may shorten its stream through the same no-clawback rule. Before the start, it may cap the end at the start and create a zero-lifetime obligation. There is no cancellation marker. Before terminal recovery, the payee may shorten it again after the shared cooldown. During terminal recovery, the payee must use the exact safe cutoff and cannot repeat for that payment. An operator may later reschedule it. A payment leaves the set only through settlement. The contract recomputes the per-asset reserve from the live payment set on every spend.

STT-policy streaming assets.
On-chain stream ingress rejects the STT script as a payout payment credential, but it does not reject the STT policy as the payment asset policy. If such a stream ever owes a positive integer quantity, it cannot make positive payment progress or leave State while that unpaid amount remains. Settlement cannot put that policy's asset at its tagged non-STT payee. Each STT token must remain at its own continuing STT output, and the spend path accepts only one input at the shared STT address. Final permanent exit and exact distribution require an empty stream list. Reserve-aware beneficiary recovery can still recover other wallet assets. The maintained dApp rejects the STT policy for fresh mint and management entries. It permits existing entries to remain forwardable and reschedulable. Custom builders must apply the same exclusion.

Streaming asset existence.
Input evidence proves that an asset exists when the payment is added. The input can belong to any address, including another holder's reference input. This check does not prove wallet ownership, sufficient payment funding, or future asset availability. One base unit can supply evidence for several new payments of the same asset. Newly minted tokens and transaction outputs alone cannot supply evidence. Existing forwarded or rescheduled payments need no new evidence. A payment asset must therefore exist before the transaction that introduces it as a stream.

Streaming reserve is point-in-time.
The reserve protects only streaming-payment value *accrued* as of a spend's upper validity bound, not the full remaining schedule. A beneficiary recovering after the owner goes silent can therefore draw down principal that a payment's *future* accrual will need, leaving later settlements underfunded. Reserving the whole forward schedule would let long-dated payments starve heirs, so the design protects only accrued value; keeping enough on hand for future accrual remains the owner's planning responsibility.

Permanent state thread.
There is no burn/close path; the small state-token deposit persists even after funds are fully recovered. This keeps the "exactly one STT, always" invariant unconditional.

Open settlement timing.
While a transaction's lower bound is before terminal recovery, any listed user, stream payee, or unlocked beneficiary may trigger a streaming payout and pay its fee. An admin may also settle. Once that lower bound reaches the final beneficiary's unlock boundary, only an admin or that beneficiary may trigger a payout. The destination and payout amount remain fixed by State. Before terminal recovery, cadence limits how often other stakeholders may settle, but the owner does not control when they choose to act. Parties with no relationship to the wallet cannot settle.

Under-funded settlement is first-come, not pro-rata.
The reserve gate is a floor on discretionary outflow, and settlement is exempt from it: a settlement's outflow is independently pinned, asset by asset, to the tagged payee outputs, so it cannot underfund anyone and needs no floor. Applying the floor to settlement was in fact a deadlock — when a wallet holds less of an asset than it owes, the floor forbids *every* outflow of that asset, including the only action that can reduce the debt, so the asset would freeze permanently for payee, operator and beneficiary alike while accrual kept growing. The exemption removes the freeze at the cost of ordering: when a wallet cannot cover everything it owes, payees are settled in the order they crank rather than proportionally. Each payment is still individually capped at its own accrual, so no payee is ever paid more than it earned; keeping the wallet funded above its accrued obligations remains the owner's planning responsibility.

Repeatable final-beneficiary recovery.
In the existing weighted withdrawal path, every beneficiary before the final one is removed after one recovery action. Removing the native-asset count cap does not let an earlier beneficiary retry an incomplete withdrawal. Ledger limits may still prevent the selected wallet inputs from fitting one transaction. The final beneficiary stays in State. It can recover separate current or future wallet UTxOs through repeated transactions. Each value-moving action may select any wallet-input set that fits ledger limits. It may leave continuing outputs for the streaming reserve or a smaller chosen withdrawal. No transaction can prove that another wallet UTxO does not exist or that no future deposit will arrive. The contract therefore has no final recovery marker or atomic all-pool guarantee. Each attempt obeys the shared thirty-minute cadence, the streaming reserve, and ledger resource limits. This path recovers wallet UTxOs only. Staking-reward withdrawal remains operator-only.

Uncadenced exact-distribution succession.
With two or more beneficiaries, exact distribution requires an empty stream list and every

beneficiary to be unlocked. The named initiating beneficiary must sign. A wallet-less form preserves State, including the cadence stamp. After acceptance, the signer can immediately build a successor against the recreated STT. Each successor spends and recreates the latest STT, so this is not byte replay. The caller supplies the fee, and no wallet value moves. A successor can invalidate another pending transaction that references the prior STT. This uncadenced succession and its STT contention are accepted design trade-offs. The sole-beneficiary form remains subject to the shared cooldown and window cap.

Exact distribution requires settled streams and divisible assets.
One underfunded stream blocks exact distribution even when it owes another asset. Additional funding or the existing reserve-aware weighted withdrawal path remains necessary. An indivisible asset cannot be split among several weighted beneficiaries. Permanent exits can leave one beneficiary, but lost keys can prevent that outcome. Exact distribution preserves rights to unselected and future deposits. It does not freeze operators, guarantee discovery of all funds, or guarantee that every one-input payout fits the ledger limits.

Authorized Consolidation controls wallet layout.
An admin, quorum, or unlocked beneficiary may preserve aggregate wallet Value while increasing the number of continuing wallet UTxOs. The ledger requires minimum ada in each output and enforces transaction byte-size and ExUnit limits. A later authorized Consolidation can reverse the split, but discovery work and cleanup fees can increase.

Value held at the STT output.
Every otherwise-valid transition may add externally funded ada to the continuing STT output. Only an admin or a multi-signature quorum using $\mathrm{RunOperator(Use)}$ may remove ada or add or remove assets under other policies. The current wallet STT itself remains fixed. A beneficiary-only State cannot recover a top-up from this output, so the top-up remains locked unless usable operator authority exists.

Only token-bearing outputs at the state-token address are spendable.
Every state token under the policy shares one address. A UTxO sent there without a state token fails the token check when spent alone. Spending it alongside the real state token fails the single-input-at-this-address check. Anyone can send such outputs to the public address. Their value is inaccessible under this validator. Accumulated outputs can burden a client that enumerates the address. The reference builder fetches the chosen transaction's outputs and checks the exact State token. This keeps unrelated address outputs out of that lookup.

Self-addressed streams.
A stream may name the wallet's own full address on-chain. Mint and stream addition exclude the state-token payment credential but do not exclude the wallet payment credential. The maintained dApp rejects this credential at mint and for new stream additions. Existing entries remain payable and removable. A custom builder can still submit the accepted on-chain configuration. When settlement consumes wallet UTxOs, a tagged output at the wallet address counts as both the configured payout and a continuing wallet output. For ADA, the continuing wallet aggregate must equal the wallet input aggregate minus the validated payout delta. ADA routing permits ADA at all correctly tagged configured stream outputs and checks only the aggregate tagged ADA amount. The exact wallet outflow can therefore reach another configured stream payee. The validators do not bind individual lovelace from one stream's delta to

that stream's address. A wallet-less settlement can use external value to create a tagged wallet output because the wallet validator does not run.

For a native asset, exact tagged-quantity matching still applies. It does not prove external delivery when the wallet is the configured payee. The acceptance test starts with two wallet inputs holding 100 tokens. It creates a tagged 10-token wallet output, an untagged 80-token wallet output, and a 10-token burn. The wallet aggregate falls from 100 to 90 while the tagged quantity remains equal to the payout delta. The Epora validators accept this transaction context. The test uses a zero fee and does not execute a minting policy or Cardano phase-one validation. A ledger-balanced form of this native-only fixture needs external ADA for a positive fee and a minting policy that accepts the burn. No external payee receives the burned quantity. Total wallet loss remains capped by the validated payout delta. These self-addressed outcomes are accepted configuration risks.

External script payout compatibility.

A streaming or exact beneficiary payout may target an unrelated script address. The payout carries the protocol's inline OutputId datum. Epora validates the complete address, tag, and amount. It cannot validate whether the receiving script later accepts that datum. The person who configures the payout must verify recipient compatibility. Another Epora wallet is compatible because its spending validator ignores input datums.

ADA settlement granularity and fee funding.

The payout-integrity routing (Section 7) forbids *any* non-protocol output from carrying the payout asset, and every Cardano output must carry a minimum amount of ADA. For an ADA-denominated stream the payout asset *is* ADA, with two consequences. First, a settlement funded from wallet value cannot emit the cranker's own fee-change output — that untagged change output would carry the payout asset and be rejected — so the cranker's funding input has only two legal ADA sinks, the tagged payee output and the transaction fee, and is allocated across the two with no residual returned (the fee is *not* equal to the input once a top-up is present: part of the input tops the payee output up to the min-UTxO amount described next, and the remainder pays the fee). Second, the settled amount can be smaller than the per-output ADA minimum, and an exact-delta payee output would then be unconstructible. To keep small ADA settlements possible, the tagged-output check for ADA requires the payee outputs to sum to *at least* the delta rather than exactly it (Section 7, "Payout integrity"). For a payout address that does not share the wallet payment credential, the cranker tops the payee's tagged output up to a valid amount with its own ADA. The wallet's net outflow remains pinned to exactly the delta and can reach only configured payees. In that non-self-addressed case, any surplus above the delta comes from the cranker. The self-addressed case above is the source-attribution exception. Its tagged wallet continuation can satisfy the stream floor while the exact wallet delta reaches another configured stream output. The residual cost is per-payee exactness across simultaneous ADA streams (see "Payout integrity"). Constructing the crank (no change output; sufficient top-up on small ADA settlements) is the off-chain builder's responsibility. A non-ADA stream normally avoids this minimum-output issue because its padding uses ADA. The self-address exception also applies under the inferred ledger conditions above. A tagged wallet continuation can match the exact delta while the burn supplies the net wallet outflow. The self-address limitation documents this accepted risk.

9 Illustrative Use Cases

The model composes into concrete configurations. The following are illustrative, not exhaustive.

9.1 Classical use cases

Inheritance and backup. An owner keeps unrestricted control and names one or more beneficiaries behind the dead-man-switch. While the owner transacts (or sends proof of life), beneficiaries can do nothing; if the owner goes silent past the deadline, beneficiaries recover their weighted shares. This transfers assets on death or key loss without ever sharing a seed phrase, and beneficiaries can be changed at any time, for example after marriage. Trust in beneficiaries is bounded (each takes only its share) and revocable.

Inheritance with vesting. As above, but a beneficiary also cannot unlock before a chosen date, for instance before reaching legal adulthood. This limits premature access by guardians while the owner's proof of life still gates recovery.

Shared treasury with allowances. A family or small team pools funds in one wallet: large moves require a multi-signature quorum; members hold capped daily allowances for everyday spending; salaries or vendor retainers run as streaming payments that settle automatically without anyone online; and a trusted party is named beneficiary so the treasury stays recoverable if the active signers are lost.

9.2 Threats mitigated in practice

The mechanisms of Sections 5 and 7 address two families of risk: the catastrophic loss or seizure of funds, and the everyday risks of managing them. Table 3 covers the first family, listing common ways funds are lost or seized, the configuration that contains each, and the most an adversary gains once that configuration is in place. Table 4 covers the second. Neither list is exhaustive, and every entry inherits the trust assumptions of Section 8. Daily-allowance claims in both tables require the assumptions of Theorem 3. In particular, the authorization-gate stance (G3) means these protections come from *distributing and capping authority*, not from restricting where an authorized key may send funds.

Beyond catastrophic loss, the same vocabulary addresses the everyday risks of managing funds: spending one's own holdings too freely, a slip of the hand, a convincing scam, or a shared pool any one person can move alone. Table 4 lists these, where shared control through weighted multi-signature does most of the work.

One scenario is worth spelling out, since its bound is the one users most often misread.

Physical coercion (the "wrench attack").

This daily bound requires the configuration assumptions in Theorem 3. When the person under duress holds only a spender key, the contract refuses to release more than that key's remaining daily allowance, whatever the holder is forced to sign. Reaching the bulk of the funds needs admin or multi-signature authority the coerced party does not carry, and under a quorum no single individual present can satisfy it. Because the cap is per day rather than per transaction, the bound holds even under sustained force: a captor can extract at most one daily allowance for each day the victim is held, never the bulk. Forcing more therefore means holding the victim longer for a small marginal sum, which raises the captor's exposure faster than the payout. The effect is that of a low-balance travel wallet, but enforced on one shared pool rather than by splitting funds across separate wallets. The limit (G3, Section 8) is that coercing a *sole admin*, or a whole quorum at once, is not contained on-chain; safety here comes from not carrying full authority on the key used to travel or transact day to day.

Threat	Containing configuration	Most the adversary gains
Physical coercion or kidnapping (the "$5 wrench") on an everyday key	Day-to-day use on a capped spender; owner authority held separately	One day's allowance per day held, never the bulk
Single key stolen (malware, phishing, lost device)	Weighted multi-signature over owners; everyday use on a spender	Nothing, or one day's allowance, until a quorum is also taken
Malicious dApp or wallet-drainer signature	Interaction through a low-allowance spender, not an owner key	One day's remaining allowance
Rogue insider in a shared treasury	Large moves need a quorum; members hold capped allowances	One member's daily allowance
Sudden death or incapacitation	Proof-of-life dead-man-switch with named beneficiaries	Nothing; beneficiaries recover only after the deadline
Lost seed or hardware failure	A backup key named as one's own beneficiary	Nothing; recover to the backup after the deadline
Greedy or dishonest heir or executor	Weighted-share recovery; each nonfinal beneficiary acts once	Only that heir's current weighted share; the final beneficiary owns all remaining weight
Premature access by a guardian	Beneficiary vesting delay (e.g. until adulthood)	Nothing before the vest date
Dependent left without income	Streaming payment behind a protected reserve	Cannot divert or underfund already-accrued payouts
Staking-reward or governance-vote capture via a foreign stake credential	Stake-credential pinning and the sweep diagnostic (Section 11)	Nothing; rewards, delegation, and vote stay with the wallet
Griefing the configuration to grow validation work	Capped access lists, linear-cost removal, reachability invariant	Rejects over-cap configuration changes. A decodable legacy list can shrink while each removal fits current limits

Table 3: Catastrophic loss-and-seizure scenarios and the configuration that contains each.

Risk	Containing configuration	What the configuration ensures
Impulsive or emotional moves (panic-sell, FOMO, gambling)	A daily allowance set as one's own cooling-off cap	At most one day's allowance leaves per day; a larger move takes a deliberate second step
Operational mistake on an everyday key (wrong amount or address)	Everyday use on a capped spender	A single slip cannot exceed the daily cap
Scam or social engineering of one signer	Weighted multi-signature, a second signer reviewing independently	No move until a quorum approves on its own devices, so one deceived signer is not enough
Malware or a compromised device on one signer	Multi-signature approvals gathered on independent devices	One compromised machine cannot move funds alone
Unilateral or unaccountable move of shared funds	A multi-signature quorum over treasury actions	No single member moves funds alone, and each approval leaves a shared on-chain record
A signer temporarily unavailable (travel, illness, lost device)	An m-of-n threshold with $m < n$	The wallet keeps operating with the remaining signers; total loss is still covered by the dead-man-switch

Table 4: Everyday control and human-factor risks, and the configuration that contains each.

10 Related Work

Per-day allowance with counterparty recovery. The closest published analog is the savings-wallet sketch in the Ethereum whitepaper [4]: the owner may withdraw at most 1% per day, a bank counterparty the same, the owner can shut off the bank's power, and together they may withdraw anything. This is prior art for the per-day-allowance-plus-single-counterparty pattern. It has no weighted-share or multi-beneficiary recovery, no inheritance, and no dead-man-switch; that is the space this design occupies.

Smart accounts and social recovery. Account-model smart-account standards such as ERC-4337 [12], and wallets built on them, add programmable authorization and guardian-based social recovery. They target a mutable-storage account model. Our contribution is a comparable permission-and-recovery surface expressed natively in EUTXO, where state is threaded through a token rather than mutated in place (Section 2).

On-chain multi-signature. Multi-signature wallets (for example Safe [13] on EVM chains, and Cardano's native and Plutus multi-signature) distribute trust across signers. They are a building block here, since weighted multi-signature is one authority path, but on their own they lack time-based recovery: there is no notion of owner inactivity triggering a fallback.

MPC and threshold-signature custody. MPC and threshold-signature wallets split a key across parties off-chain. They are sometimes presented as a uniformly safer default, but the evidence does not support that: zero-day vulnerabilities were found across more than fifteen production MPC/TSS implementations [14]. We therefore position an on-chain, validator-enforced model not as cryptographically superior but as differently trusted. Its rules are public, deterministic, and open to review (G4), with no off-chain signing protocol to get wrong.

11 Implementation

On-chain validators. The validators are written in Aiken [15], chosen for its developer experience, built-in testing, and fit for an open-source contract meant to be read and reviewed (G4). AI assists development, widening test coverage: it generates unit and regression tests, drafts the attack scenarios behind each invariant of Section 7, and expands property-based fuzzing inputs that probe boundary conditions a hand-written suite tends to miss. Every generated test is reviewed before it enters the suite, and the security invariants it checks are stated by us, not inferred by the model. On-chain enforcement is split as in Section 4. VERIFIED in the 2026-09-07 build: the compiled wallet validator is 9,375 bytes, or 64.60% of the 14,513-byte STT validator. The asset-evidence guard adds 247 bytes to the 2026-09-06 STT build of 14,266 bytes. The always-fail reference store is 93 bytes. These sizes measure raw blueprint code before wallet parameter application. They do not measure full transactions. Correction: the previous 14,428-byte STT size and 64.98% ratio did not match the 2026-09-06 blueprint snapshot. The Aiken suite spans unit checks, an attack-regression log, and property-based fuzzing.

Off-chain stack and interface. Everything else (building, balancing, and submitting transactions) happens off-chain, where we use Mesh [16]. A reference web interface, built with Next.js [17], connects to any CIP-30 browser wallet [18] (and, over WalletConnect [19], to mobile or remote wallets), previews every transaction before signing, and routes chain queries through a server-side proxy so no third-party API key reaches the browser. Day-to-day actions are presented as guided flows, with staking, governance, and recovery actions on an advanced surface.

Recovery preparation in the interface. Normal recovery starts with a permanent withdrawal. An explicit transaction-size or execution-limit failure offers preparation and single-pool distribution. The signed transaction size check can also trigger this offer. Funding, authorization, stale-input and generic evaluation errors retain their own error path. The interface does not retry automatically. A preparation requires a separate preview and confirmation. Its builder reads the selected inputs and State again, rejects a changed reviewed State, and checks the actual balanced output layout. Pool edits or changes to the selected funds invalidate the preview. The permanent withdrawal still warns about omitted discovered UTxOs, unused entitlement, loss of recovery rights, and fees paid by the connected wallet.

Stake-credential diagnostic. One diagnostic is a stake-credential check. It queries the wallet's *payment* credential against a public indexer. This finds funds at an unintended ("Frankenstein") or outdated stake credential that an address query would miss. The app opens the consolidation flow for these funds. One consolidation has no fixed wallet-input, continuing-output, or native-asset-row cap. It must preserve aggregate wallet Value, and each continuing output must satisfy the shared stake, datum, and reference-script guards. Ledger byte-size and ExUnit limits decide which remaining shapes fit. A caller can retry with fewer inputs or outputs if a large shape exceeds ledger limits. Final-beneficiary recovery may also consume any wallet-input set that fits ledger limits and has no five-asset cap. The indexer's public API sends no cross-origin access header, so the browser cannot query it directly. A small same-origin proxy on the application server forwards the call. The server therefore sees the queried payment credential. We accept this because the query does not work from the browser. The check runs when a wallet opens and on demand from the tools surface. Figure 6 sketches the interaction.

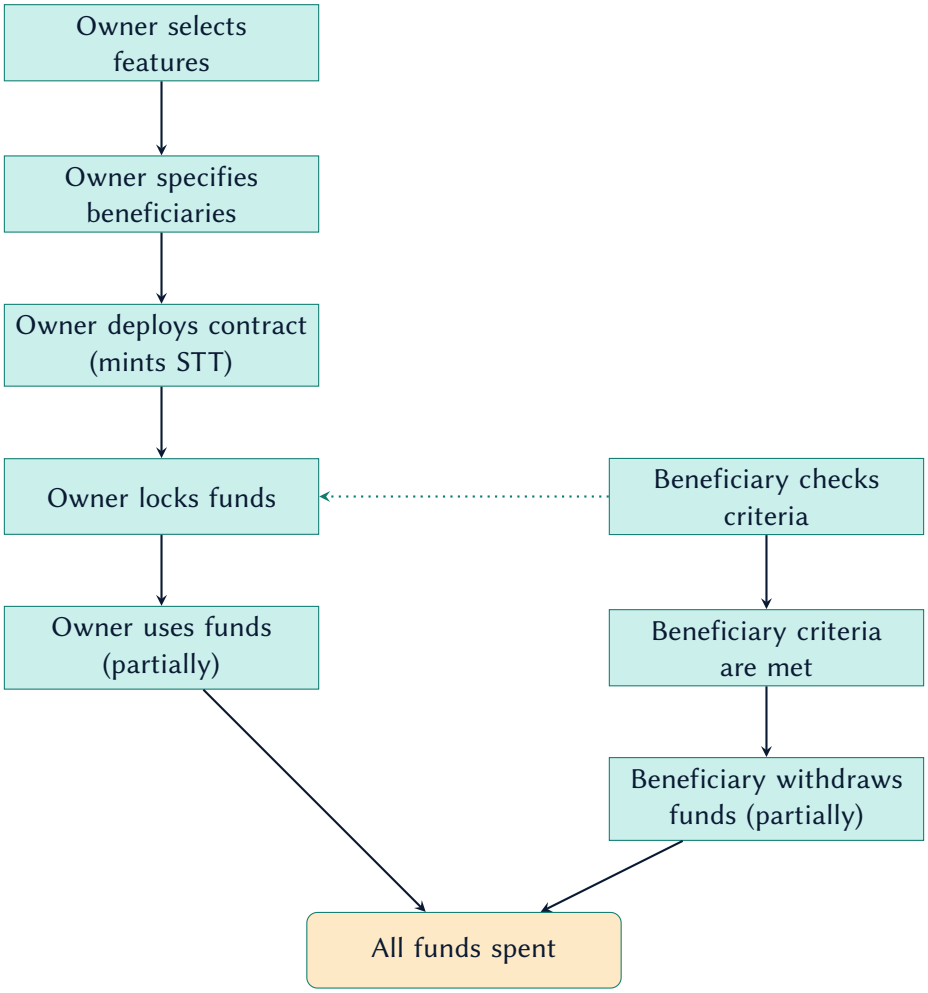

Figure 6: End-to-end interaction, from wallet creation to recovery.

12 Conclusion

This design threads a shared wallet's entire configuration through a single state-thread token and bounds every movement of funds with a two-validator handshake. Spending limits, weighted multi-signature, a proof-of-life dead-man-switch, weighted-share multi-beneficiary recovery, and stakeholder-settled streaming payments are then *configuration* on one reviewable EUTXO contract, while custody stays with the owner throughout. The security argument is small enough to review in full: one composed guarantee (Section 4), an invariant defended per asset (Section 7), and the limits and trust assumptions of Section 8. The contracts and the reference interface are open source.

Status and future work. The implementation currently targets the Cardano Preprod test network, and the validators have not had an external audit, so the wallet should not yet hold real funds. The remaining work is tracked publicly through the project's Catalyst milestones [2].

Acknowledgments

This work was funded by Project Catalyst [1] under Fund 11 (Cardano Use Cases: Concept). We thank the Cardano community for supporting it.

References

[1] Project Catalyst, "(Dead-man-switch) Permission based Wallet — Fund 11, Cardano Use Cases: Concept," https://projectcatalyst.io/funds/11/cardano-use-cases-concept/dead-man-switch-permission-based-wallet, 2024.

[2] ——, "Milestone module — project 1100002," https://milestones.projectcatalyst.io/projects/1100002/milestones, 2024.

[3] M. M. T. Chakravarty, J. Chapman, K. MacKenzie, O. Melkonian, M. Peyton Jones, and P. Wadler, "The Extended UTXO Model," in *Financial Cryptography and Data Security Workshops*, 2020, https://iohk.io/en/research/library/papers/the-extended-utxo-model/.

[4] V. Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," 2014, https://ethereum.org/en/whitepaper/.

[5] National Cyber Security Centre, "Protocol Design Principles," https://www.ncsc.gov.uk/paper/protocol-design-principles, 2020.

[6] G. Almashaqbeh, A. Bishop, and J. Cappos, "ABC: A Cryptocurrency-Focused Threat Modeling Framework," in *IEEE INFOCOM Workshops (CryBlock)*, 2019, https://arxiv.org/abs/1903.03422.

[7] Cardano Developer Portal, "Native Assets and Tokens," https://developers.cardano.org/docs/learn/core-concepts/assets/, 2024.

[8] Aiken, "Common Design Patterns — State Thread Token and Multivalidators," https://aiken-lang.org/fundamentals/common-design-patterns, 2024.

[9] Plutonomicon contributors, "The State Thread Token (STT) Pattern," https://plutonomicon.github.io/plutonomicon/statethread, 2022.

[10] Cardano Developer Portal, "The Extended UTxO (EUTXO) Model," https://developers.cardano.org/docs/get-started/technical-concepts/eutxo/, 2024.

[11] Cardano Foundation, "CIP-0019: Cardano Addresses," https://cips.cardano.org/cip/CIP-19, 2024.

[12] V. Buterin, Y. Weiss, D. Tirosh *et al.*, "ERC-4337: Account Abstraction Using Alt Mempool," https://eips.ethereum.org/EIPS/eip-4337, 2021.

[13] Safe Ecosystem Foundation, "Safe: Smart Account Infrastructure," https://safe.global/, 2024.

[14] Fireblocks Cryptography Research, "BitForge: Zero-Day Vulnerabilities in Multiple MPC/TSS Wallet Implementations," https://www.fireblocks.com/blog/bitforge-fireblocks-researchers-uncover-vulnerabilities-in-over-15-major-wallet-providers, 2023.

[15] Aiken, "Aiken — a modern smart contract platform for Cardano," https://aiken-lang.org/, 2024.

[16] Mesh, "Mesh — an open-source library for building Web3 apps on Cardano," https://meshjs.dev/, 2024.

[17] Vercel, "Next.js — the React framework for the web," https://nextjs.org/, 2024.

[18] Cardano Foundation, "CIP-0030: Cardano dApp-Wallet Web Bridge," https://cips.cardano.org/cip/CIP-30, 2021.

[19] WalletConnect, "WalletConnect v2.0 Protocol Specifications," https://specs.walletconnect.com/2.0, 2024.